

分子动力学模拟

一、实验目的

模拟粒子的运动：通过迭代计算，根据粒子的初始位置和速度，模拟粒子在空间中的运动。粒子的位置和速度会在每个时间步长进行更新，从而模拟其运动轨迹。

计算粒子的势能：使用 Lennard-Jones 势能模型，根据粒子之间的距离计算粒子的势能。Lennard-Jones 势能模型是一种常用的描述粒子间相互作用的模型，通过计算粒子间的排斥力和吸引力来模拟粒子的相互作用。

计算粒子的动能：根据粒子的质量和速度，计算粒子的动能。动能反映了粒子运动的快慢程度，它与粒子的质量和速度的平方成正比。

研究能量变化：通过记录模拟过程中的动能、势能和总能量，绘制能量随时间的变化曲线。这可以帮助我们了解粒子系统的能量分布和变化趋势，从而对系统的稳定性和动力学行为进行分析。

观察粒子轨迹：绘制粒子在空间中的轨迹，可以直观地观察粒子的运动路径和变化趋势。这有助于理解粒子的运动行为和相互作用模式。

通过这些实验目的，我们可以深入了解分子系统的动力学特性，包括粒子的运动规律、相互作用模式、能量变化趋势等。这对于研究和理解分子结构、材料性质、化学反应等领域具有重要意义，也为设计和优化分子系统提供了有价值的参考。

二、实验内容

初始设置：通过设定粒子数量、模拟空间的盒子大小、粒子初始位置和速度等参数来进行初始设置。可以根据具体需求设定粒子数量和盒子大小，同时随机生成粒子的初始位置和速度。

动力学模拟：使用分子动力学方法进行模拟。在每个时间步长内，通过计算粒子间的相互作用力和随机力来更新粒子的速度和位置。具体步骤包括：

计算粒子间距离：对于每对粒子，计算它们之间的距离，并根据周期性边界条件调整距离。

计算势能和力：根据 Lennard-Jones 势能模型，计算粒子间的势能和力。根据粒子间的距离，通过 Lennard-Jones 势能公式计算势能值，并计算出对应的力。

更新力和速度：根据粒子的质量和力的作用，更新粒子的速度。

添加随机力：引入随机力以模拟温度效应。通过生成服从高斯分布的随机数，并按照一定的比例加入到粒子速度中，从而模拟粒子的随机热运动。

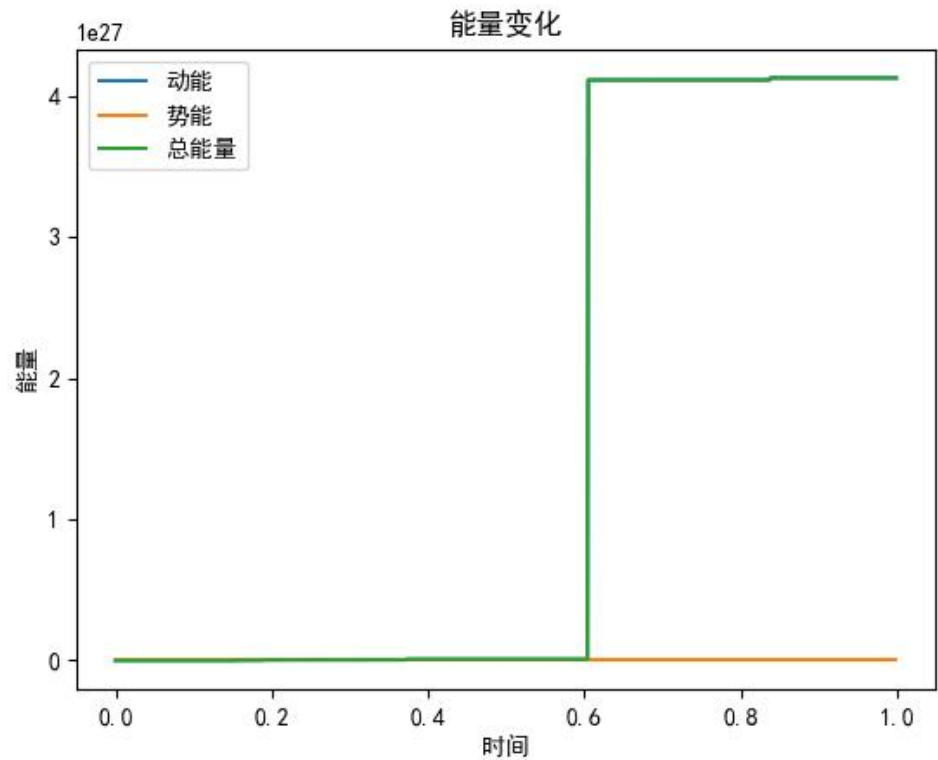
更新位置：根据粒子的速度和时间步长，更新粒子的位置。

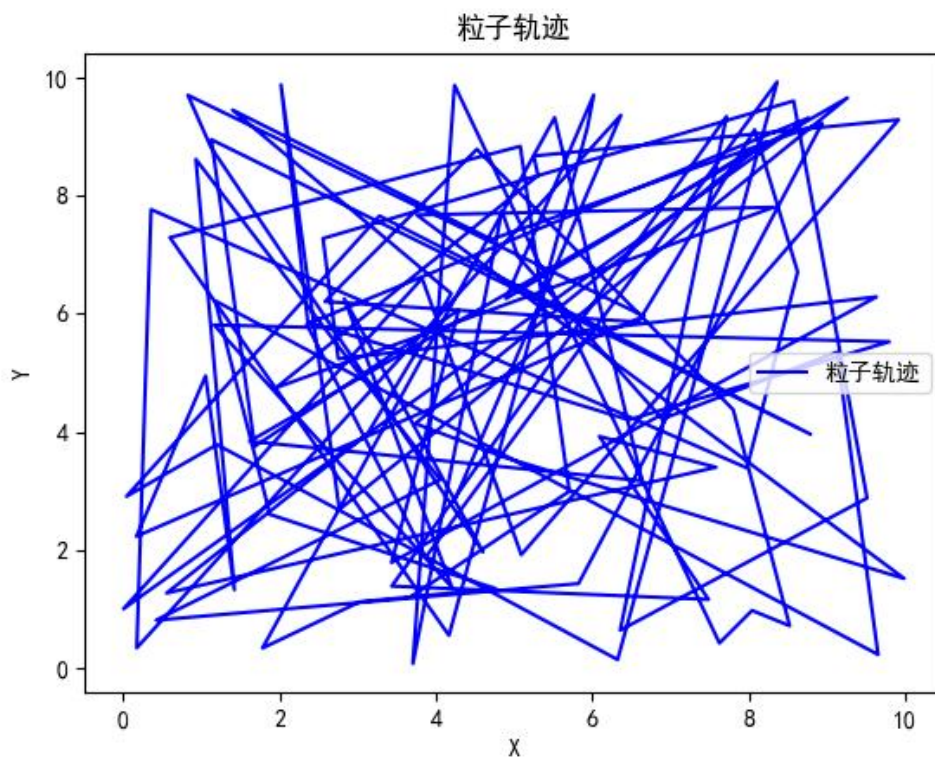
边界条件处理：应用周期性边界条件，使得粒子在盒子边界处穿过边界时能够回到另一边界，保持模拟系统的连续性。

计算能量：在每个时间步长内，计算粒子的动能和势能。动能计算使用粒子质量和速度的平方和，势能计算使用 Lennard-Jones 势能公式。

能量变化分析：通过记录模拟过程中的动能、势能和总能量，绘制能量随时间的变化曲线。这可以帮助观察系统的能量变化趋势，分析系统的稳定性和动力学行为。

粒子轨迹观察：绘制粒子在三维空间中的轨迹。通过绘制粒子位置的演化，可以直观地观察粒子的运动路径和变化趋势。





三、 实验代码

```
import numpy as np

import matplotlib.pyplot as plt

import MDAnalysis as mda

plt.rcParams['font.sans-serif'] = ['SimHei']

def calculate_forces(positions, box_length):

    forces = np.zeros_like(positions)

    # 计算势能和力

    for i in range(len(positions)):
```

```

for j in range(i + 1, len(positions)):

    # 计算粒子间距离

    r = positions[j] - positions[i]

    r -= np.round(r / box_length) * box_length

    dist_squared = np.sum(r ** 2)

    # 避免除以零或无效值

    if dist_squared != 0:

        # Lennard-Jones 势能和力

        r_6 = dist_squared ** 3

        r_12 = r_6 ** 2

        potential = 4 * (1 / r_12 - 1 / r_6)

        force = 48 * (1 / r_12 - 0.5 / r_6) * r

        # 更新总力

        forces[i] += force

        forces[j] -= force

    return forces

def simulate_dynamics(positions, velocities, box_length, num_steps, dt, temperature):

```

```
num_particles = len(positions)

masses = np.ones(num_particles) # 假设所有粒子质量相同

# 初始化动能和势能数组

kinetic_energies = np.zeros(num_steps)

potential_energies = np.zeros(num_steps)


for step in range(num_steps):

    # 计算力

    forces = calculate_forces(positions, box_length)


    # 更新速度

    velocities += forces / masses[:, np.newaxis] * dt


    # 随机力

    random_forces = np.random.randn(num_particles, 3) * np.sqrt(2 * temperature / dt)

    velocities += random_forces / np.sqrt(masses[:, np.newaxis])


    # 更新位置

    positions += velocities * dt


    # 边界条件：周期性边界条件
```

```

positions -= np.floor(positions / box_length) * box_length

# 计算动能

kinetic_energies[step] = 0.5 * np.sum(masses * np.sum(velocities ** 2, axis=1))

# 计算势能

norm_positions = np.linalg.norm(positions, axis=1)

valid_indices = np.where(norm_positions != 0) # 排除为零的情况

potential_energies[step] = np.sum(

    4 * ((1 / (norm_positions[valid_indices] ** 12)) - (1 /

(norm_positions[valid_indices] ** 6))))

return positions, velocities, kinetic_energies, potential_energies

# 定义粒子位置

num_particles = 100

box_length = 10.0

positions = np.random.uniform(low=0.0, high=box_length, size=(num_particles, 3))

# 初始化速度

velocities = np.random.randn(num_particles, 3) # 高斯分布随机速度

```

```
# 模拟参数

num_steps = 1000

dt = 0.001

temperature = 300 # 模拟温度，单位：K

# 进行模拟

final_positions, final_velocities, kinetic_energies, potential_energies =
simulate_dynamics(positions, velocities,

box_length, num_steps, dt,

temperature)

# 绘制能量变化

total_energies = kinetic_energies + potential_energies

time = np.arange(num_steps) * dt

plt.figure()

plt.plot(time, kinetic_energies, label='动能')

plt.plot(time, potential_energies, label='势能')

plt.plot(time, total_energies, label='总能量')

plt.xlabel('时间')
```

```
plt.ylabel('能量')

plt.title('能量变化')

plt.legend()

plt.show()


# 绘制粒子轨迹

plt.figure()

plt.plot(final_positions[:, 0], final_positions[:, 1], 'b-', label='粒子轨迹')

plt.xlabel('X')

plt.ylabel('Y')

plt.title('粒子轨迹')

plt.legend()

plt.show()
```